

CS223 Computer Architecture & Organization

Optimization Techniques in Cache Memory-1



John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Accessing Cache Memory



$$\text{Average memory access time} = \text{Hit time} + (\text{Miss rate} \times \text{Miss penalty})$$

- ❖ **Hit Time:** Time to find the block in the cache and return it to processor [*indexing, tag comparison, transfer*].
- ❖ **Miss Rate:** Fraction of cache access that result in a miss.
- ❖ **Miss Penalty:** Number of additional cycles required upon encountering a miss to fetch a block from the next level of memory hierarchy.

How to optimize cache ?

- ❖ Reduce Average Memory Access Time
- ❖ $AMAT = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$
- ❖ Motives
 - ❖ Reducing the miss rate
 - ❖ Reducing the miss penalty
 - ❖ Reducing the hit time

Larger Block Size

- ❖ **Larger block size to reduce miss rate**

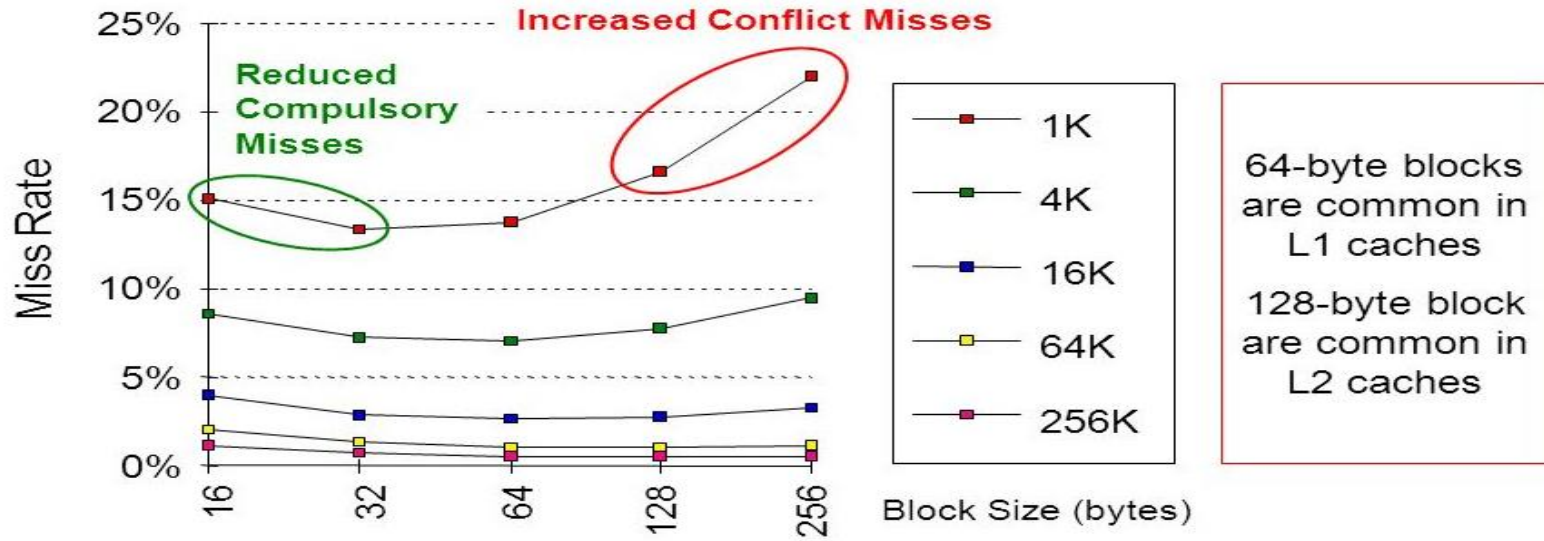
- ❖ **Advantages**

- ❖ Utilize spatial locality
- ❖ Reduces compulsory misses

- ❖ **Disadvantages**

- ❖ Increases miss penalty
- ❖ More time to fetch a block to the cache [bus width issue]
- ❖ Increases conflict misses
- ❖ More number of blocks will be mapped to the same location
- ❖ May bring useless data and evict useful data [pollution]

Larger Block Size



Larger Caches

- ❖ **Larger cache to reduce miss rate**

- ❖ **Advantages**

 - ❖ Reduces capacity misses

 - ❖ Can accommodate larger memory footprint

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

- ❖ **Drawbacks**

 - ❖ Longer hit time

 - ❖ Higher cost, area and power

Larger Caches



Higher Associativity

❖ Higher associativity to reduce miss rate

- ❖ Fully associative caches are the best, but high hit time.
- ❖ So increase the associativity to an optimal possible level

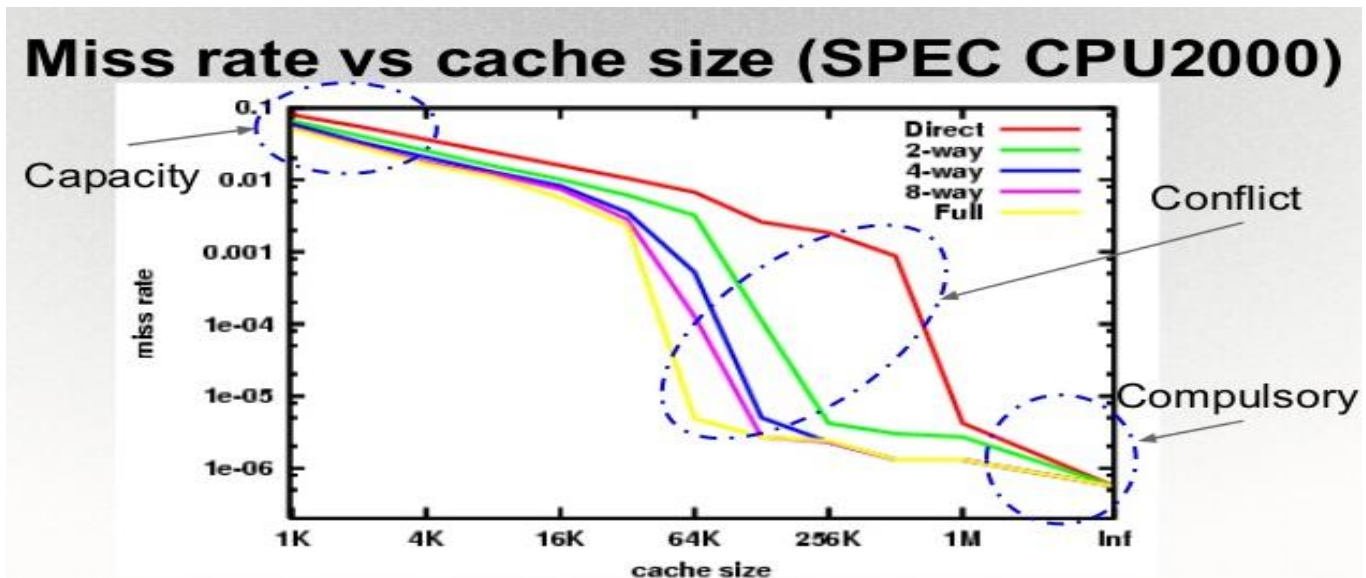
❖ Advantages

- ❖ Reduce conflict miss
- ❖ Reduce miss rate and eviction rate

❖ Drawbacks

- ❖ Increase in the hit time
- ❖ Complex design than direct mapped
- ❖ More time to search in the set (tag comparison time)

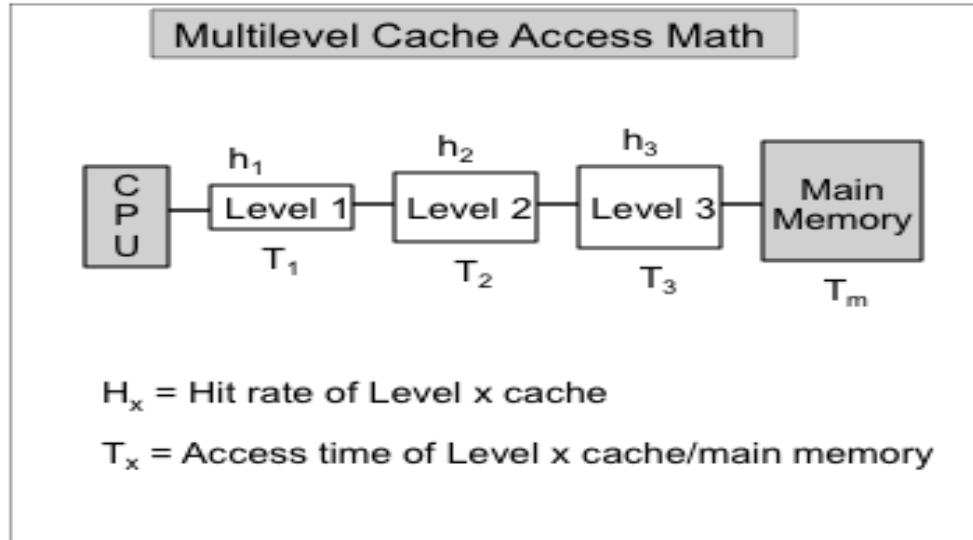
AMAT vs cache associativity



Multilevel caches

- ❖ **Multilevel caches to reduce miss penalty**
- ❖ **Caches should be faster** to keep pace with the speed of processors, **AND cache should be larger** to overcome the widening gap between the processor and main memory
- ❖ Add another level of cache between the cache and memory.
- ❖ The first-level cache (L1) can be small enough to match the clock cycle time of the fast processor. [Low hit time]
- ❖ The second-level cache (L2) can be large enough to capture many accesses that would go to main memory, thereby lessening the effective miss penalty. [Low miss rate]

Multilevel caches



Average memory access time = Hit time_{L1} + Miss rate_{L1} × Miss penalty_{L1}

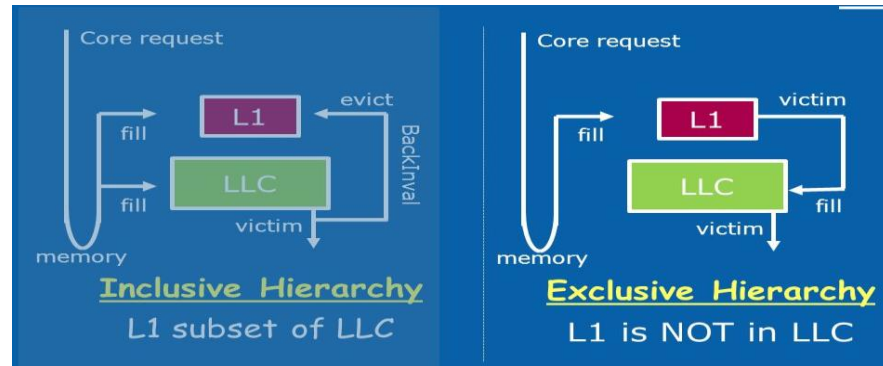
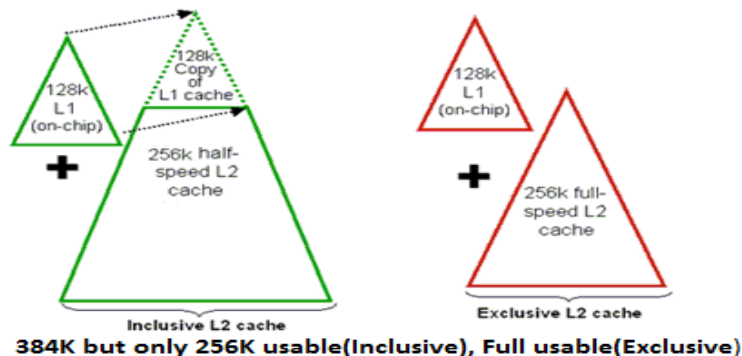
Miss penalty_{L1} = Hit time_{L2} + Miss rate_{L2} × Miss penalty_{L2}

Average memory access time = Hit time_{L1} + Miss rate_{L1}
× (Hit time_{L2} + Miss rate_{L2} × Miss penalty_{L2})

Multilevel caches

- ❖ **Multilevel caches to reduce miss penalty**
- ❖ **Local miss rate:** Number of misses in a cache level divided by number of memory access to this level.
- ❖ **Global miss rate:** Number of misses in a cache level divided by number of memory access generated by the CPU.
- ❖ **Inclusive and Exclusive caches**

Cache Architecture Comparisons



Prioritize read miss over writes

- ❖ **Prioritize read misses to reduce miss penalty**
- ❖ If a read miss has to evict a dirty memory block, the normal sequence is write the dirty block to memory and read the missed block

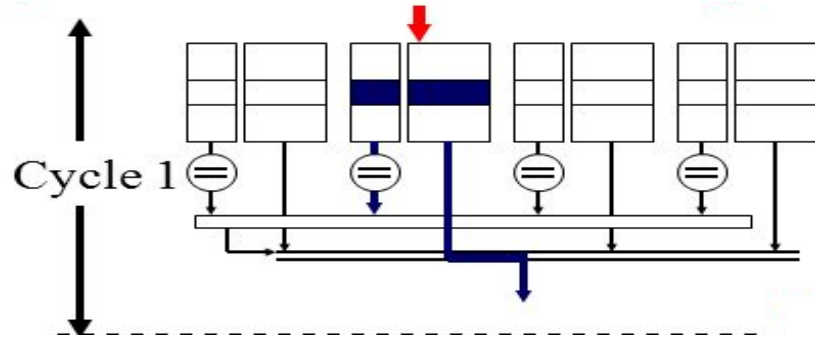
```
SW R3, 512(R0)      ;M[512] ← R3      (cache index 0)
LW R1, 1024(R0)     ;R1 ← M[1024]    (cache index 0)
LW R2, 512(R0)      ;R2 ← M[512]    (cache index 0)
```

- ❖ Optimization: copy the dirty block to a buffer, read from memory and then write the block - reduces CPU's waiting time on read miss

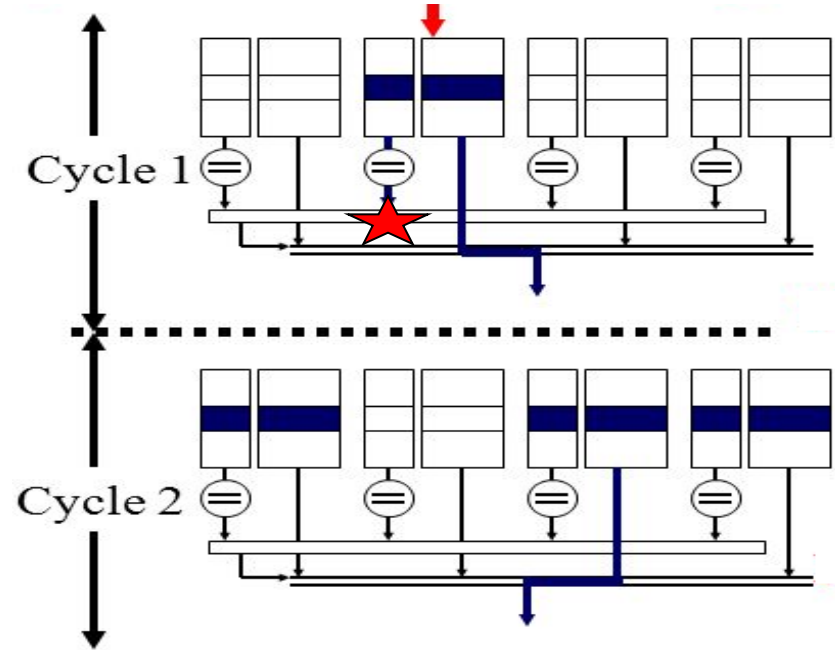
Way Prediction

- ❖ **Predict the way in a set to reduce hit time**
- ❖ To improve hit time, predict the way to pre-set multiplexer
- ❖ Extra bits are set to predict the block with in the set
- ❖ Remember the MRU way of a given set
- ❖ Using the prediction bits, power gating can be done on unused ways for reducing power.

Way Prediction



Way Prediction - Correct

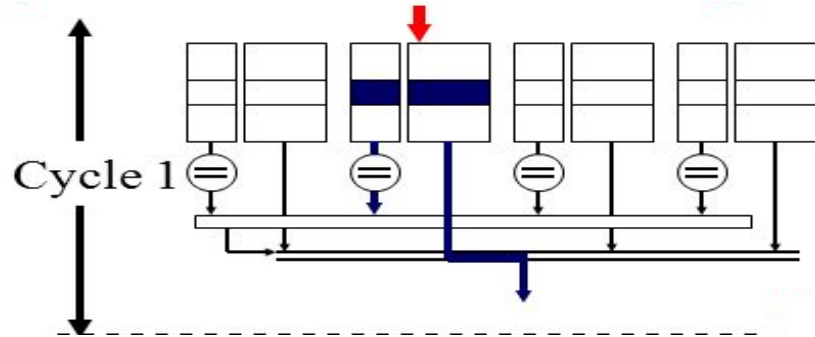


Way Prediction Miss

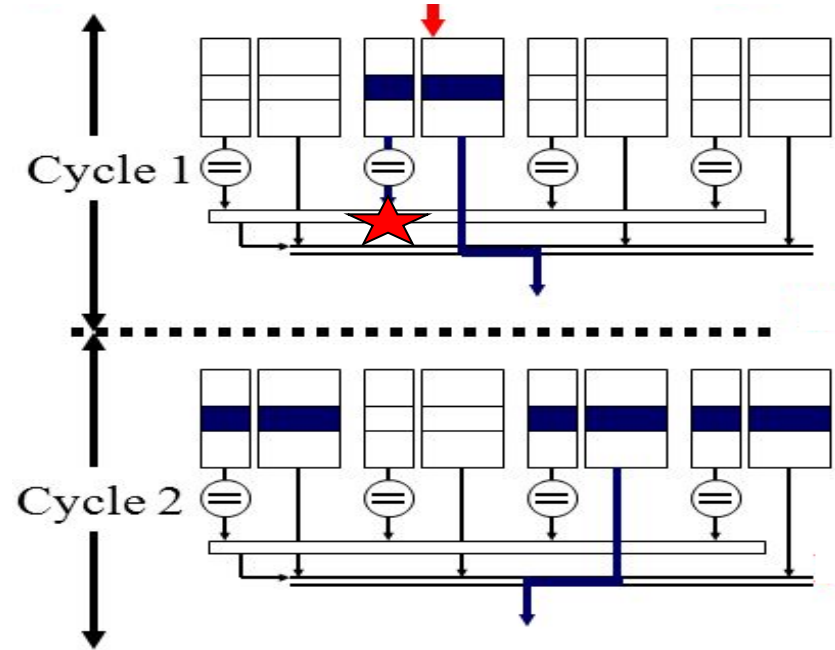
Way Prediction

- ❖ **Predict the way in a set to reduce hit time**
- ❖ To improve hit time, predict the way to pre-set multiplexer
- ❖ Extra bits are set to predict the block with in the set
- ❖ Remember the MRU way of a given set
- ❖ Using the prediction bits, power gating can be done on unused ways for reducing power.

Way Prediction



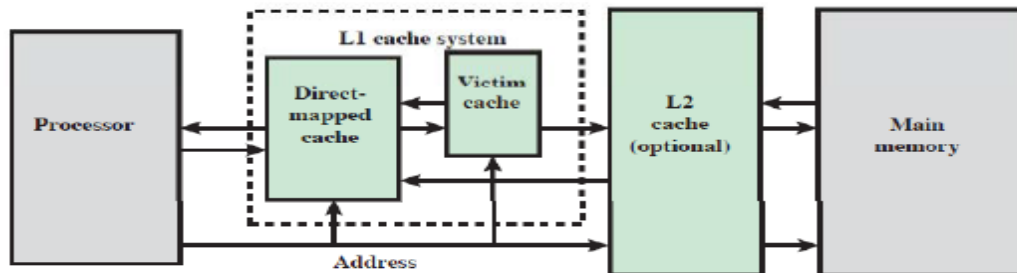
Way Prediction - Correct



Way Prediction Miss

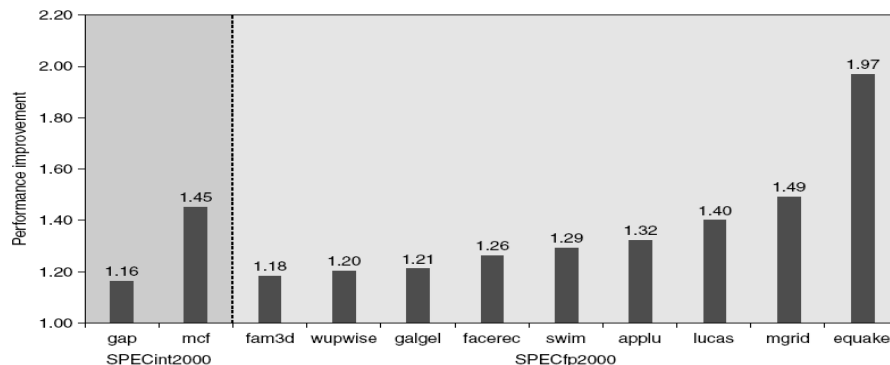
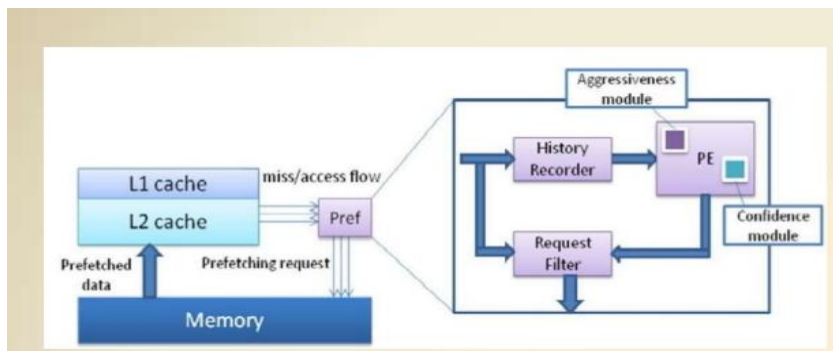
Victim Caches

- ❖ **Introduce victim caches to reduce mis- penalty**
- ❖ Additional storage near L1 for MR evicted blocks
- ❖ Efficient for thrashing problem in L1 cache
- ❖ Look up Victim Caches before L2
- ❖ L1 and Victim caches are exclusive



Hardware Prefetching

- ❖ **Pre-fetching to reduce miss rate and miss penalty.**
- ❖ Pre-fetch items before processor request them.
- ❖ Fetch more blocks on miss -include next sequential block
- ❖ Requested block is kept in I-cache and next in stream buffer.
- ❖ If a missed block is in stream buffer, cache miss is cancelled





johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>